

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

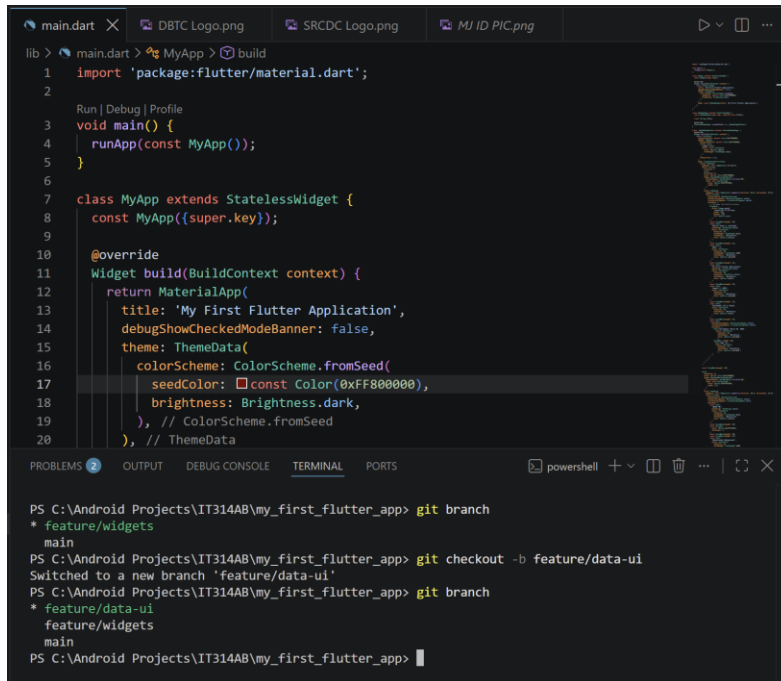
Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch



```
lib > main.dart > MyApp > build
1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp(const MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8   const MyApp({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return MaterialApp(
13      title: 'My First Flutter Application',
14      debugShowCheckedModeBanner: false,
15      theme: ThemeData(
16        colorScheme: ColorScheme.fromSeed(
17          seedColor: const Color(0xFF800000),
18          brightness: Brightness.dark,
19        ), // ColorScheme.fromSeed
20      ), // ThemeData
21    );
22  }
23 }
```

```
PS C:\Android Projects\IT314AB\my_first_flutter_app> git branch
* feature/widgets
main
PS C:\Android Projects\IT314AB\my_first_flutter_app> git checkout -b feature/data-ui
Switched to a new branch 'feature/data-ui'
PS C:\Android Projects\IT314AB\my_first_flutter_app> git branch
* feature/data-ui
feature/widgets
main
PS C:\Android Projects\IT314AB\my_first_flutter_app>
```

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Screenshot:

- Your identified data
- The corresponding data types

Profile Information	Identified Data	Corresponding Data Types
Name	'Martin Jonas A. Aranzado'	string

Course & Section	'BSIT - 3'	string
Age	21	int
Hobbies	{'watching movies', 'playing games', 'mahjong'}	set
Height	152.4	double
Student Status	true	bool

Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

Create variables for at least:

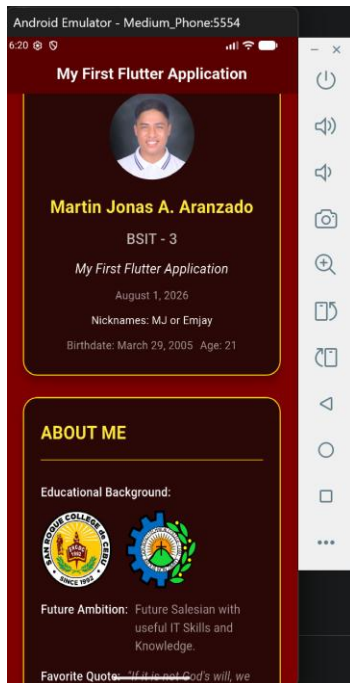
- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

Screenshot:

- Your variables
- Their data types
- Application still running

```
class _MyHomePageState extends State<MyHomePage> {
  String name = 'Martín Jonas A. Aranzado';
  String courseAndsection = 'BSIT - 3';
  int age = 21;
  Set<String> hobbies = {'watching movies', 'playing games', 'mahjong'};
  double height = 152.4;
  bool studentStatus = true;
}
```



🚩 Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

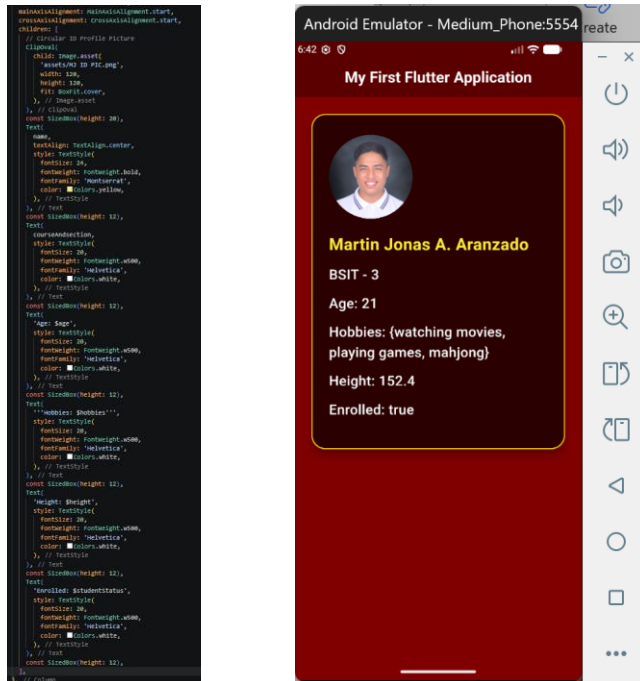
Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables
- Updated application

```
class _MyHomePageState extends State<MyHomePage> {  
  String name = 'Martin Jonas A. Aranzado';  
  String courseAndsection = 'BSIT - 3';  
  int age = 21;  
  Set<String> hobbies = {'watching movies', 'playing games', 'mahjong'};  
  double height = 152.4;  
  bool studentStatus = true;  
}
```



Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

Then change the variable to another registered image.

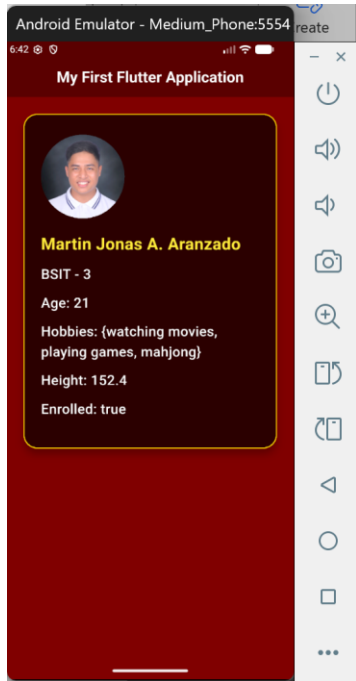
The image displayed by the application should change.

Screenshot:

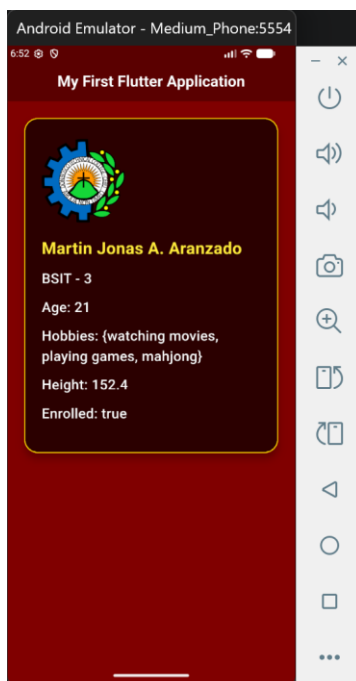
- Image variable
- Image widget using the variable
- Updated application

```
class _MyHomePageState extends State<MyHomePage> {
  String name = 'Martin Jonas A. Aranzado';
  String courseAndSection = 'BSIT - 3';
  int age = 21;
  Set<String> hobbies = {'watching movies', 'playing games', 'mahjong'};
  double height = 152.4;
  bool studentStatus = true;
  String imagePath = 'assets/MJ ID PIC.png';
}
```

```
ClipOval(
  child: Image.asset(
    imagePath,
    width: 120,
    height: 120,
    fit: BoxFit.cover,
  ), // Image.asset
), // ClipOval
```



```
class _MyHomePageState extends State<MyHomePage> {
  String name = 'Martin Jonas A. Aranzado';
  String courseAndSection = 'BSIT - 3';
  int age = 21;
  Set<String> hobbies = {'watching movies', 'playing games', 'mahjong'};
  double height = 152.4;
  bool studentStatus = true;
  String imagePath = 'assets/DBTC Logo.png';
}
```





Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data

```
class _MyHomePageState extends State<MyHomePage> {  
  List<Profile> profiles = [  
    Profile(  
      imagePath: 'assets/IronMan Logo.png',  
      name: 'Iron Man',  
      courseAndSection: 'BS Engineering - 4',  
      age: 23,  
      hobbies: ['Building Suits', 'AI management'],  
      height: 185.2,  
      studentStatus: true,  
    ),  
    Profile(  
      imagePath: 'assets/Spiderman Logo.png',  
      name: 'Spiderman',  
      courseAndSection: 'BSIT - 2',  
      age: 19,  
      hobbies: ['Photography', 'Web Slinging'],  
      height: 175.2,  
      studentStatus: true,  
    ),  
    Profile(  
      imagePath: 'assets/Batman Logo.png',  
      name: 'Batman',  
      courseAndSection: 'BS Criminology - 4',  
      age: 24,  
      hobbies: ['Martial Arts', 'Learning new languages'],  
      height: 186.5,  
      studentStatus: false,  
    ),  
    Profile(  
      imagePath: 'assets/Superman Logo.png',  
      name: 'Superman',  
      courseAndSection: 'BA Journalism- 3',  
      age: 22,  
      hobbies: ['Reporting', 'Farming'],  
      height: 187.0,  
      studentStatus: true,  
    ),  
    Profile(  
      imagePath: 'assets/Captain America Logo.png',  
      name: 'Captain America',  
      courseAndSection: 'BS Military Science- 4',  
      age: 30,  
      hobbies: ['Boxing', 'Drawing'],  
      height: 189.0,  
      studentStatus: false,  
    ),  
  ],  
};
```



Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

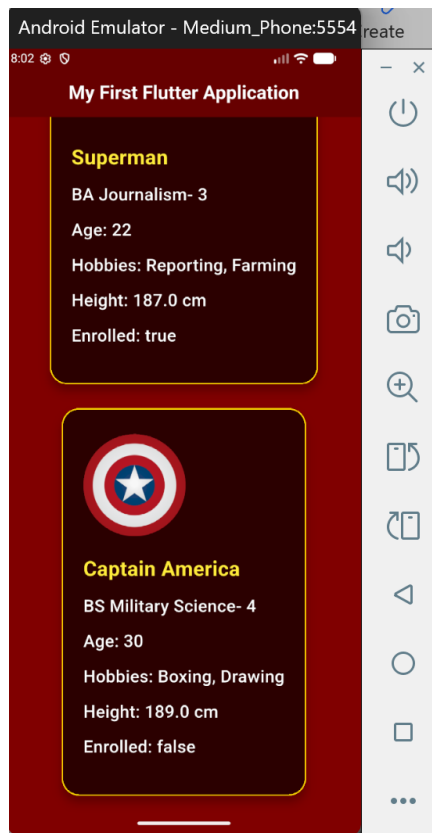
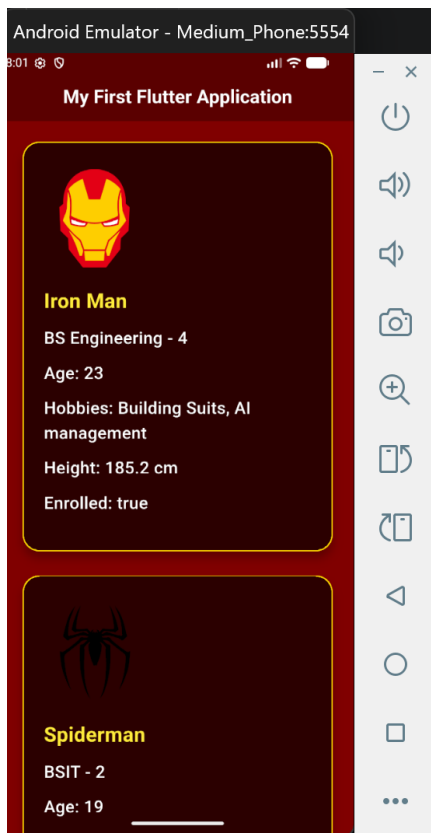
Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

- The scrolling widget in your code
- Top portion of the application
- The rest of the portion of the application showing the later profiles

```
), // AppBar  
body: SingleChildScrollView(  
  child: Padding(  
    padding: const EdgeInsets.all(16.0),  
  ),  
),
```



Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: **missing**

Profile B

Name: Juan

Course: **missing**

Hobby: Gaming

Profile C

Name: **missing**

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

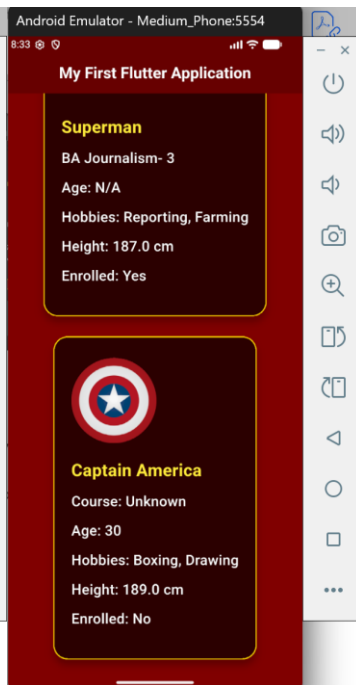
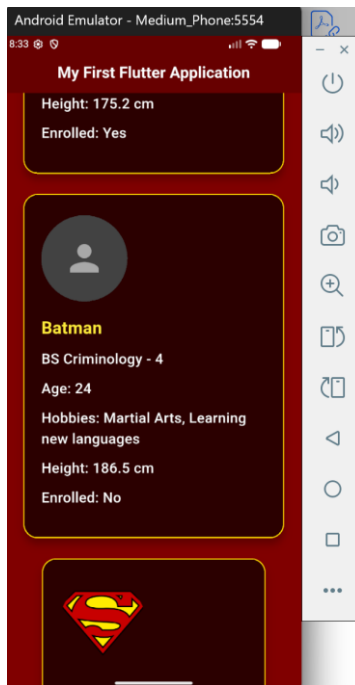
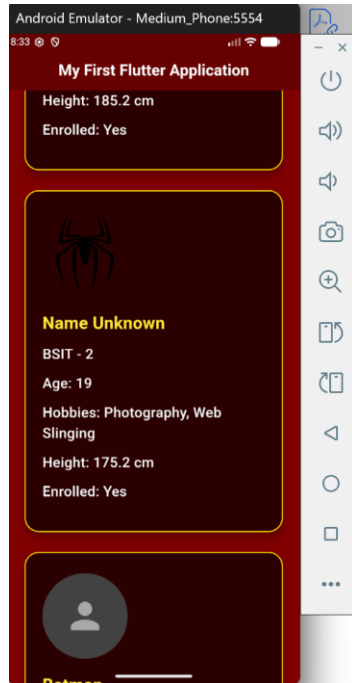
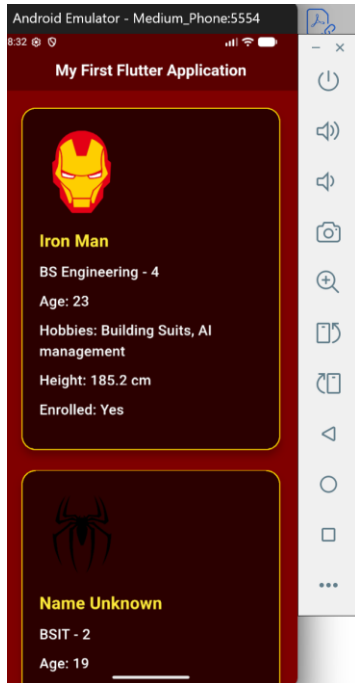
- Incomplete profile data
- Application displaying fallback values
- Application still functioning correctly

```
List<Profile> profiles = [
    Profile(
        imagePath: 'assets/IronMan Logo.png',
        name: 'Iron Man',
        courseAndSection: 'BS Engineering - 4',
        age: 23,
        hobbies: ['Building Suits','AI management'],
        height: 185.2,
        studentStatus: true,
    ),
    Profile(
        imagePath: 'assets/Spiderman Logo.png',
        courseAndSection: 'BSIT - 2',
        age: 19,
        hobbies: ['Photography','Web Slinging'],
        height: 175.2,
        studentStatus: true,
    ),
    Profile(
        name: 'Batman',
        courseAndSection: 'BS Criminology - 4',
        age: 24,
        hobbies: ['Martial Arts','Learning new languages'],
        height: 186.5,
        studentStatus: false,
    ),
    Profile(
        imagePath: 'assets/Superman Logo.png',
        name: 'Superman',
        courseAndSection: 'BA Journalism- 3',
        hobbies: ['Reporting','Farming'],
        height: 187.0,
        studentStatus: true,
    ),
    Profile(
        imagePath: 'assets/Captain America Logo.png',
        name: 'Captain America',
        age: 30,
        hobbies: ['Boxing','Drawing'],
        height: 189.0,
        studentStatus: false,
    ),
];

class Profile {
  String? imagePath;
  String? name;
  String? courseAndSection;
  int? age;
  List<String>? hobbies;
  double? height;
  bool? studentStatus;

  Profile({
    String? imagePath,
    String? name,
    String? courseAndSection,
    int? age,
    List<String>? hobbies,
    double? height,
    bool? studentStatus,
  }) {
    this.imagePath = imagePath;
    this.name = name;
    this.courseAndSection = courseAndSection;
    this.age = age;
    this.hobbies = hobbies;
    this.height = height;
    this.studentStatus = studentStatus;
  }

  void print() {
    print('Profile: $name, $courseAndSection, $age, $height, $studentStatus');
  }
}
```





Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

feature/data-ui

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
MJ ARANZADO@LAPTOP-U8N5E77G MINGW64 /c/Android Projects/IT314AB (feature/data-ui)
$ git commit -m "feat: added variables for data to be input and allows for null data"
[feature/data-ui 2ba8403] feat: added variables for data to be input and allows for null data
6 files changed, 114 insertions(+), 160 deletions(-)
create mode 100644 my_first_flutter_app/assets/Batman Logo.png
create mode 100644 my_first_flutter_app/assets/Captain America Logo.png
create mode 100644 my_first_flutter_app/assets/IronMan Logo.png
create mode 100644 my_first_flutter_app/assets/Spiderman Logo.png
create mode 100644 my_first_flutter_app/assets/Superman Logo.png

MJ ARANZADO@LAPTOP-U8N5E77G MINGW64 /c/Android Projects/IT314AB (feature/data-ui)
$ git push origin -u feature/data-ui
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 18 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 6.51 MiB | 496.00 KiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/data-ui' on GitHub by visiting:
remote:   https://github.com/EmJayAA/IT314AB_Aranzado/pull/new/feature/data-ui
remote:
To https://github.com/EmJayAA/IT314AB_Aranzado.git
 * [new branch]      feature/data-ui -> feature/data-ui
branch 'feature/data-ui' set up to track 'origin/feature/data-ui'.

MJ ARANZADO@LAPTOP-U8N5E77G MINGW64 /c/Android Projects/IT314AB (feature/data-ui)
```

